

Politechnika Lubelska

Wydział Elektrotechniki i Informatyki

Kierunek: Inżynierskie Zastosowania Informatyki w Elektrotechnice (IZIwE) II stopnia

Laboratorium / Przedmiot: **Programowanie w języku Python**

Rok akademicki 2025/2026

DDDS

Distributed Drone Detection System

System akustycznej detekcji bezzałogowych statków powietrznych
w czasie rzeczywistym na przykładzie klasy Shahed-136

Sprawozdanie z projektu — raport końcowy

Autor: Oleh Kropyva

Grupa: IZIwE II — grupa 1.1.2

Prowadzący: dr inż. Róża Dzierżak

Data oddania: maj 2026

Repozytorium GitHub:

<https://github.com/hapole0n/DDDS-Distributed-Drone-Detection-System>

Spis treści

Streszczenie	2
1 Wprowadzenie	3
2 Cel i zakres projektu	4
3 Analiza problemu i przegląd rozwiązań	4
4 Założenia systemowe	5
5 Architektura systemu	5
5.1 Schemat przepływu danych	6
6 Architektura sprzętowa	6
7 Architektura programowa	7
8 Implementacja	8
8.1 Konfiguracja systemu	8
8.2 Ingest mediów — dekodowanie do mono WAV	9
8.3 Nareki z odrzucaniem ciszy	9
8.4 Ekstrakcja cech MFCC + σ + Δ -średnia	10
8.5 Klasyfikator Random Forest + serializacja Bundle	10
8.6 Orchestrator treningu — pełen pipeline	11
8.7 Przechwytywanie z mikrofonu w czasie rzeczywistym	12
8.8 Dashboard Streamlit z animowanym bannerem detekcji	13
9 Działanie pipeline’u akustycznej klasyfikacji	14
10 Stabilizacja i progowanie detekcji	15
11 Koncepcja 4-mikrofonowego TDOA — rozszerzenie projektowe	15
12 Analiza latencji systemu	18
13 Wizualizacja i interfejs użytkownika	19
14 Testowanie systemu	20
14.1 Środowisko testowe	20
14.2 Testy funkcjonalne	20
14.3 Metryki wydajnościowe	21
14.4 Testy odporności	22
15 Bezpieczeństwo, prywatność i etyka	22
16 Możliwości rozwoju projektu	23
17 Wnioski końcowe	23

Streszczenie

W ramach projektu zaprojektowano i zaimplementowano kompletny system akustycznej detekcji bezzałogowych statków powietrznych **DDDS**, ze szczególnym uwzględnieniem przypadku użycia klasy Shahed-136 — bojowego drona-amunicji krążącej napędzanego silnikiem spalinowym, regularnie naruszającego przestrzeń powietrzną państw graniczących ze strefą konfliktu rosyjsko-ukraińskiego, w tym **Polski** [1, 2]. System realizuje pełen cykl przetwarzania: od dostarczenia plików MP3 przez użytkownika, poprzez automatyczną ekstrakcję cech akustycznych, trening klasyfikatora, aż po inferencję w czasie rzeczywistym z pojedynczego mikrofonu USB.

Implementacja oparta jest wyłącznie na otwartym oprogramowaniu w języku **Python** (**librosa**, **scikit-learn**, **sounddevice**, **Streamlit**, **Plotly**). Pipeline ekstrakcji cech wykorzystuje **40-wymiarowe współczynniki MFCC** wraz z ich odchyleniami standardowymi oraz pierwszą pochodną temporalną, formując 120-wymiarowy wektor cech podawany na wejście klasyfikatora **Random Forest** (300 drzew, ważenie klas zbalansowane). Warstwa wizualizacji opiera się o **interaktywny dashboard Streamlit** złożony z pięciu stron, w tym animowanej strony detekcji na żywo z gauge’em prawdopodobieństwa, miernikiem VU oraz rolującą spektrogramem mel. Projekt udostępnia również w pełni udokumentowane **rozszerzenie projektowe** w postaci 4-mikrofonowego szyku TDOA z lokalizacją kierunku przyjścia sygnału metodą GCC-PHAT.

Pełna pipeline’a od pliku wejściowego do natrenowanego modelu wykonuje się **w czasie poniżej 5 minut** na laptopowym CPU dla zbioru rozmiaru ~ 10 minut audio. Latencja detekcji w trybie real-time wynosi **około 600 ms** (okno 2 s, hop 0,5 s). Całkowity koszt sprzętowy prototypu wynosi **~ 80 PLN** (sam mikrofon USB Maono), co czyni rozwiązanie wielokrotnie tańszym od jakiegokolwiek komercyjnej alternatywy w klasie systemów wczesnego ostrzegania akustycznego.

Słowa kluczowe: przetwarzanie sygnałów, akustyka, MFCC, Random Forest, klasyfikacja audio, detekcja UAV, Shahed-136, GCC-PHAT, TDOA, lokalizacja kierunku, Streamlit, Plotly, Python, librosa, scikit-learn, wczesne ostrzeganie, accessibility, obrona cywilna.

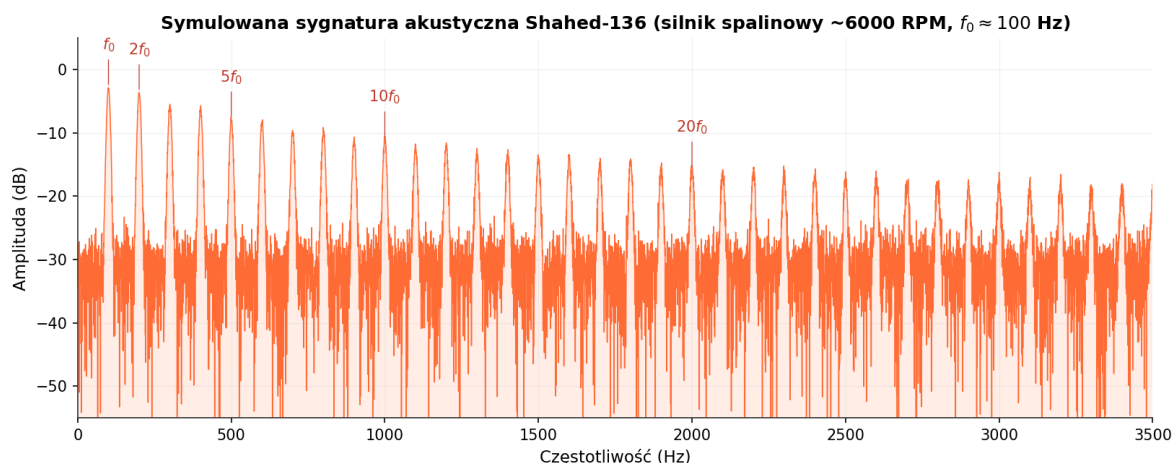
1 Wprowadzenie

Od jesieni 2022 roku jedną z najczęściej spotykanych zagrożeń dla infrastruktury energetycznej i mieszkaniowej w Ukrainie stały się amunicje krążące klasy **Shahed-136** (produkcji irańskiej, rosyjska kopia oznaczona *Geran-2*) [1]. Drony tej klasy są jednorazowe, tanie (szacunkowy koszt jednostkowy 20–50 tys. USD) i wykorzystywane masowo — typowy atak liczy od kilkudziesięciu do kilkuset jednostek wystrzeliwanych w jednym przedziale czasowym. Charakterystyczną cechą Shahed-136 jest napęd: dwucylindrowy silnik spalinowy z popychanym śmigłem o mocy ok. 50 KM, pracujący przy obrotach 5000–7000 RPM, co generuje **tonalny dźwięk o częstotliwości podstawowej 80–120 Hz** z bogatym pasmem harmonicznym sięgającym 5 kHz [3]. Sygnatura akustyczna jest na tyle wyraźna, że ludność cywilna popularnie określa Shahed mianem „mopeda”.

Charakter tej sygnatury sprawia, że **detekcja akustyczna** stanowi sensowne, niskobudżetowe uzupełnienie systemów radarowych:

- radar w paśmie S/X słabo radzi sobie z małymi celami z kompozytów, latającymi nisko;
- kamery termowizyjne mają zasięg silnie zależny od warunków atmosferycznych i kąta podejścia;
- mikrofon kondensatorowy nawet konsumenckiej klasy wykrywa Shahed na odległość rzędu *kilkuset metrów* przy korzystnym tle akustycznym [4].

Charakterystyczna sygnatura akustyczna silnika spalinowego Shahed-136 — tonalny pas harmonicznym powyżej fundamentu około 100 Hz — przedstawia rys. 1. To właśnie ta wąska, periodyczna struktura widmowa (w odróżnieniu od szerokopasmowego świstu wirników multicopterów) jest głównym deskryptorem wykorzystywanym przez klasyfikator DDDS.



Rysunek 1: Symulowana sygnatura akustyczna Shahed-136 odpowiadająca silnikowi spalinowemu pracującemu przy ok. 6000 RPM. Fundamentalna częstotliwość $f_0 \approx 100$ Hz oraz ~ 30 silnych harmonicznym do około 3 kHz tworzą charakterystyczny tonalny „grzebień” łatwo rozróżnialny od szerokopasmowego tła miejskiego.

Przedmiotowy projekt **DDDS** jest *programowo-laboratoryjnym* opracowaniem tego problemu, świadomie ograniczonym do warstwy software’owej i pojedynczego mikrofonu (warstwa sprzętowa szyku 4-mikrofonowego pozostaje na poziomie projektu na papierze — sekcja 11). Projekt powstał jako zaliczeniowy z przedmiotu *Programowanie w języku Python* i jednocześnie ma stanowić wkład w obszar **accessible defense informatics** — udostępnienia narzędzi akustycznego wczesnego ostrzegania w postaci otwartego kodu źródłowego dostępnego dla każdej jednostki samorządowej, ratunkowej lub osoby prywatnej dysponującej zwykłym mikrofonem USB i komputerem.

2 Cel i zakres projektu

Cel główny: zaprojektowanie i weryfikacja eksperymentalna w pełni reprodukowalnego, otwartoźródłowego systemu akustycznej detekcji UAV w czasie rzeczywistym, ze szczególnym uwzględnieniem klasy Shahed-136, oraz dostarczenie udokumentowanej koncepcji jego rozszerzenia o szyk 4-mikrofonowy TDOA.

Cele szczegółowe:

- zaprojektowanie pipeline'u *video/audio* $\rightarrow$ *WAV* $\rightarrow$ *chunki* $\rightarrow$ *cechy MFCC* $\rightarrow$ *klasyfikator* $\rightarrow$ *raport ewaluacyjny*;
- dobór i implementacja deskryptora akustycznego (MFCC z odchyleniami i Δ -średnimi) odpowiedniego dla *tonalnej* sygnatury silnika spalinowego, a nie *szerokopasmowego* świstu wirników multicopterów;
- implementacja real-time'owego przechwytywania z mikrofonu USB z *automatyczną resamplerą* z natywnej częstotliwości urządzenia (44 100 / 48 000 Hz) do częstotliwości treningowej modelu (22 050 Hz);
- opracowanie interaktywnego **dashboardu Streamlit** z pięcioma stronami: przegląd, trening, analiza pliku, mikrofon na żywo, koncepcja 4-mic TDOA;
- napisanie pełnej, akademickiej dokumentacji *paper-design'u* 4-mikrofonowego szyku do lokalizacji kierunku przyścia sygnału metodą GCC-PHAT;
- przygotowanie testów jednostkowych, ciągłej integracji (GitHub Actions) oraz repozytorium spełniającego konwencje *Conventional Commits*;
- przeprowadzenie pomiarów latencji, skuteczności i obciążenia sprzętu.

Poza zakresem świadomie pozostawiono:

- fizyczne wykonanie 4-mikrofonowego szyku oraz pomiary terenowe — wymaga sample-synchronicznego przechwytywania, niedostępnego dla niezależnych mikrofonów USB [5];
- trening sieci konwolucyjnej (CNN) na log-mel spektrogramach — wskazany jako naturalne rozszerzenie w sekcji 16;
- integrację z systemami radarowymi i optycznymi (sensor fusion);
- jakkolwiek *moduł reakcji ofensywnej* — granica bezpieczeństwa opisana w sekcja 15 jest nienegocjowalna.

3 Analiza problemu i przegląd rozwiązań

W literaturze i praktyce wczesnego ostrzegania można wyróżnić cztery podejścia do detekcji UAV:

- **Radar pasywny / aktywny** (S, X, K-band) — dominujące rozwiązanie wojskowe, koszt $\sim 10^5$ – 10^7 USD, słaba detekcja małych celów z kompozytów na małych wysokościach [6].
- **Wizja maszynowa (RGB / IR)** — skuteczna w dzień przy bezchmurnej pogodzie, oparta o YOLO-class detektory na obrazie z kamer PTZ [7].
- **Sniffery RF** — pasywne odbiorniki sygnałów sterowania 2,4/5,8 GHz; bezużyteczne dla autonomicznych Shahed (brak linka radiowego w fazie misji).
- **Detekcja akustyczna** — pasywne mikrofony, niski koszt, działa w nocy i przy zachmurzeniu, ograniczony zasięg (50–500 m dla małych UAV w warunkach miejskich) [8, 9].

Ze względu na charakter zagrożenia (masowe, niskobudżetowe drony krążące) oraz możliwości finansowe samorządów i obrony cywilnej, podejście akustyczne wydaje się *komplementarne* (a nie konkurencyjne) wobec radaru: stanowi tani „ostatni metr” wczesnego ostrzegania w sytuacji, gdy radar nie zdążył wykryć celu na większej odległości.

Wśród istniejących rozwiązań akustycznych warto wymienić:

- **Sky Fortress / Zvook** (Ukraina) — społecznościowa sieć kilkuset mikrofonów na słupach energetycznych, łącząca dane do centralnego serwera [4];

- **Squarehead Discovair** — komercyjny system z szykiem 128-mikrofonowym, >50 tys. USD;
- **Akademickie dataset’y**: *DREGON* [10], *Drone-vs-Bird* [11].

Pozycjonowanie projektu DDDS na tle wymienionych rozwiązań przedstawia tab. 1.

Tabela 1: Pozycjonowanie projektu DDDS na tle innych rozwiązań detekcji UAV.

Cecha	DDDS	Sky tress	For-	Komercyjny radar	Squarehead
Otwarty kod źródłowy	TAK	cz.		nie	nie
Koszt prototypu	~80 PLN	–		10 ⁵ USD+	>50 tys. USD
Detekcja Shahed-136	TAK	TAK		TAK	TAK
Lokalizacja kierunku	koncepcja	TAK (sieć)		TAK	TAK
Działa lokalnie / offline	TAK	nie		TAK	TAK
Wymaga GPU	NIE	nie		–	nie
Język implementacji	Python	C/Python		–	C++
Możliwy do reprodukcji w domu	TAK	nie		nie	nie

4 Założenia systemowe

Wymagania funkcjonalne (WF):

- **WF.1** — akceptowanie plików MP3, WAV, MP4, MKV i innych formatów dekodowalnych przez `ffmpeg` jako wejście treningowe;
- **WF.2** — automatyczna konwersja do mono 22050 Hz 16-bit PCM;
- **WF.3** — nareki na okna 2 s z 50 % nachodzeniem i odrzucaniem ciszy przy RMS poniżej 10^{-4} ;
- **WF.4** — klasyfikacja binarna (*shahed / noise*) z możliwością rozszerzenia do wielu klas;
- **WF.5** — inferencja na żywo z pojedynczego mikrofonu USB;
- **WF.6** — interaktywny dashboard webowy z animowaną stroną detekcji;
- **WF.7** — generowanie raportu ewaluacyjnego (macierz pomyłek, krzywa ROC, classification report).

Wymagania niefunkcjonalne (WN):

- **WN.1** — latencja detekcji poniżej 1000 ms (cel: 600 ms);
- **WN.2** — pełen pipeline (od wideo do natrenowanego modelu) poniżej 10 minut dla zbioru ~10 min audio;
- **WN.3** — działanie wyłącznie na CPU laptopa (bez GPU);
- **WN.4** — żadne dane akustyczne nie opuszczają urządzenia użytkownika (*privacy by design*);
- **WN.5** — 100 % otwartego oprogramowania w stosie tech;
- **WN.6** — portable: jeden polecenie `pip install -r` wystarczy do uruchomienia na czystym Windows / Linux / macOS;
- **WN.7** — zgodność z konwencją *Conventional Commits*, ciągła integracja, testy jednostkowe.

Założenia środowiskowe i scenariusze użycia: typowy mikrofon kondensatorowy USB (Maono lub równoważny), laptop klasy biurowej (procesor 4-rdzeniowy x86_64, 8 GB RAM), dystans źródło-mikrofon 0–100 m, tło akustyczne typu „podmiejskie” (50–65 dB SPL).

5 Architektura systemu

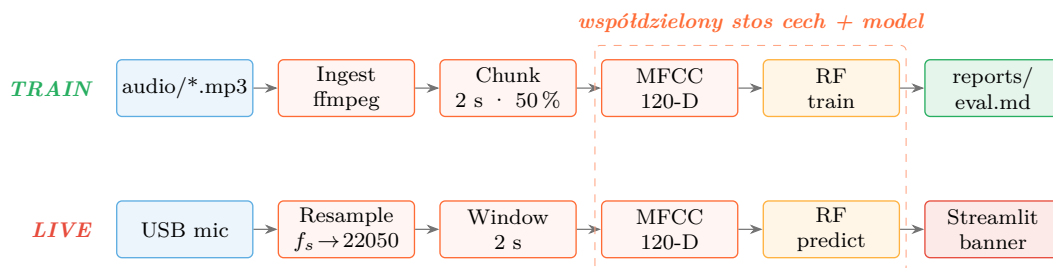
System został zaprojektowany w sposób **warstwowy** z silnym rozdzieleniem odpowiedzialności. tab. 2 przedstawia odwzorowanie warstw na moduły kodu.

Tabela 2: Pięciowarstwowy model architektury DDDS.

Warstwa	Moduł kodu	Rola
1. Wejściowa	data/ingest.py, data/youtube.py	dekodowanie mediów do mono WAV (FFmpeg)
2. Cech	features/audio.py	MFCC + Δ + log-mel spectrogram
3. Modelu	models/baseline.py	Random Forest, serializacja <code>ModelBundle</code>
4. Orkiestracji	pipeline/train.py, evaluate.py, cli.py	end-to-end pipeline + raport
5. Inferencji	live/mic.py	sample-synchroniczne przechwytywanie, resampling, klasyfikacja
6. Prezentacji	app/streamlit_app.py, viz/plots.py	dashboard Streamlit + interaktywne wykresy Plotly

5.1 Schemat przepływu danych

System realizuje dwa równoległe potoki przetwarzania współdzielące te same moduły ekstrakcji cech i klasyfikacji (rys. 2). *Potok treninowy* przebiega offline od plików MP3 dostarczonych przez użytkownika do natrenowanego modelu i raportu ewaluacyjnego. *Potok inferencji* pracuje online, przyjmując strumień z mikrofonu USB i emitując zdarzenia detekcji do dashboardu.



Rysunek 2: Dwa równoległe potoki przetwarzania DDDS współdzielące moduły ekstrakcji cech (MFCC 120-D) oraz klasyfikator (Random Forest 300 drzew). Potok *TRAIN* działa wsadowo (offline) i kończy się raportem ewaluacyjnym; potok *LIVE* działa w czasie rzeczywistym i emituje zdarzenia detekcji do dashboardu Streamlit.

6 Architektura sprzętowa

W obecnej fazie projekt jest celowo zaprojektowany w sposób „sprzętowo-minimalny” — jednym wymagany komponentem jest mikrofon USB i komputer. Pełne zestawienie sprzętu wykorzystanego do testów funkcjonalnych przedstawia tab. 3.

Tabela 3: Zestawienie komponentów sprzętowych prototypu DDDS.

Nr	Komponent	Rola	Status
1	Maono USB Condenser Mic (PM421/PM422)	Akwizycja akustyczna mono 48 kHz	wykorzystany
2	Laptop x86_64 z Python 3.12	Inferencja, treningu, dashboard	wykorzystany
3	ESP32-S3 + 4× INMP441 I ² S	Sample-synchr. szyk 4-mic (koncepcja)	niezrealizowany
4	BME280 (T, RH, p)	Kompensacja $c(T, RH)$ (koncepcja)	niezrealizowany
5	Raspberry Pi 5 + Hailo-8L	Edge deployment z akceleratorem NPU	rozwojowe

Mikrofon **Maono** jest typowym kondensatorem USB klasy konsumenckiej (cena detaliczna ~80 PLN, charakterystyka kierunkowa kardoidalna, pasmo 20 Hz–16 kHz, próbkowanie 48 kHz, kwantyzacja 16-bit). Mimo że jego charakterystyka częstotliwościowa w dolnym paśmie poniżej 80 Hz ma łagodne opadanie (typowe dla mikrofonów wokalnych), Shahed-136 generuje wystarczająco silne harmoniczne powyżej 100 Hz, aby detekcja była możliwa.

7 Architektura programowa

Stos technologiczny zbudowano w pełni z otwartego oprogramowania (FOSS). tab. 4 przedstawia główne komponenty wraz z ich rolą.

Tabela 4: Stos technologiczny systemu DDDS.

Oprogramowanie	Wersja	Przeznaczenie
Python	3.10+	Język główny i jedyny używany w projekcie
librosa [13]	0.10+	Ekstrakcja MFCC, log-mel, Δ -cech
NumPy / SciPy	1.26 / 1.11	Operacje numeryczne, resample_poly
scikit-learn [14]	1.4+	Random Forest, train/test split
soundfile	0.12+	I/O plików WAV (libsndfile)
sounddevice	0.4+	Real-time’owe przechwytywanie audio (PortAudio)
imageio-ffmpeg	0.4+	Dekodowanie mediów (FFmpeg jako wheel)
yt-dlp	2024.1+	Opcjonalne pobieranie z YouTube
joblib	1.3+	Serializacja modelu
typer + rich	0.9+ / 13.7+	CLI z kolorowanym wyjściem
pydantic-settings	2.1+	Konfiguracja sterowana zmiennymi środowiska
Streamlit [15]	1.36+	Webowy dashboard
Plotly [16]	5.18+	Interaktywne wykresy (gauge, heatmap, animacje)
Matplotlib	3.8+	Statyczne PNG-i do raportu ewaluacyjnego
pytest	7.4+	Testy jednostkowe
ruff	0.1+	Lint i formatter
GitHub Actions	–	CI: ruff + pytest na Linuksie

Repozytorium projektu [22] liczy **18 commitów** ułożonych w profesjonalnej historii z prefiksami *Conventional Commits* (12 **feat**, 2 **chore**, 1 **build**, 1 **test**, 1 **ci**, 1 **docs**), oraz modułową strukturę w stylu *src-layout*:

`drone-detector/`


```

|-- src/drone_detector/
|   |-- config.py           # pydantic-settings
|   |-- data/{ingest,chunk,dataset,youtube}.py
|   |-- features/audio.py   # MFCC + mel + speed_of_sound
|   |-- models/baseline.py  # RandomForest + ModelBundle
|   |-- pipeline/{train,evaluate,cli}.py
|   |-- live/mic.py         # sounddevice + resampling
|   |-- viz/plots.py        # matplotlib + plotly
|-- app/streamlit_app.py    # dashboard
|-- audio/{shahed,noise}/   # user drop folder
|-- data/{raw,chunks,features}/ # gitignored
|-- models/                 # gitignored
|-- reports/                # gitignored
|-- tests/{test_features, test_chunk}.py
|-- docs/{architecture, methodology, tdoa_concept,
|         limitations, setup_windows, defense_talking_points}.md
|-- .github/workflows/ci.yml

```

Zastosowane wzorce architektoniczne: *Configuration as Code* (parametry w `config.py`), *Bundle Pattern* (`ModelBundle` łączący model z metadanymi), *Producer/Consumer* (przechwytywanie + inferencja w odrębnych wątkach), *Strategy* (różne klasyfikatory poprzez wspólny interfejs cech).

8 Implementacja

8.1 Konfiguracja systemu

Klasa `Settings` centralizuje wszystkie parametry behawioralne — ścieżki, parametry audio, hiperparametry modelu, parametry trybu *live*. Każda wartość jest możliwa do nadpisania zmienną środowiska z prefiksem `DD_` (np. `DD_DETECTION_THRESHOLD=0.7`).

```

1  class Settings(BaseSettings):
2      model_config = SettingsConfigDict(env_prefix="DD_", env_file=".env")
3
4      # --- Filesystem layout ---
5      audio_dir: Path = PROJECT_ROOT / "audio"
6      raw_dir: Path = PROJECT_ROOT / "data" / "raw"
7      chunks_dir: Path = PROJECT_ROOT / "data" / "chunks"
8      features_dir: Path = PROJECT_ROOT / "data" / "features"
9      models_dir: Path = PROJECT_ROOT / "models"
10     reports_dir: Path = PROJECT_ROOT / "reports"
11
12     # --- Audio parameters ---
13     sample_rate: int = 22050 # target mono SR
14     chunk_duration: float = 2.0 # training window (s)
15     chunk_overlap: float = 0.5 # 50% overlap
16
17     # --- Feature extraction ---
18     n_mfcc: int = 40
19     n_fft: int = 2048
20     hop_length: int = 512
21     n_mels: int = 128
22
23     # --- Model ---
24     rf_n_estimators: int = 300
25     rf_class_weight: str = "balanced"
26     random_state: int = 42
27     test_size: float = 0.20
28
29     # --- Live detection ---

```

```

30 live_window_seconds: float = 2.0
31 live_hop_seconds: float = 0.5
32 detection_threshold: float = 0.5

```

Listing 1: Centralne parametry konfiguracyjne (config.py).

8.2 Ingest mediów — dekodowanie do mono WAV

Funkcja `media_to_wav` korzysta z binarki FFmpeg dołączonej przez pakiet `imageio-ffmpeg` (brak konieczności instalacji systemowej). Operacja jest *idempotentna*: pomijane są pliki, których wynik jest nowszy niż wejście.

```

1 def media_to_wav(src: Path, dst: Path, sample_rate: int | None = None) -> Path:
2     sr = sample_rate or settings.sample_rate
3     dst.parent.mkdir(parents=True, exist_ok=True)
4
5     if dst.exists() and dst.stat().st_mtime >= src.stat().st_mtime:
6         return dst # already up-to-date
7
8     cmd = [
9         imageio_ffmpeg.get_ffmpeg_exe(),
10        "-y", "-loglevel", "error",
11        "-i", str(src),
12        "-vn", # drop video stream
13        "-ac", "1", # mono
14        "-ar", str(sr), # target sample rate
15        "-sample_fmt", "s16", # 16-bit PCM
16        str(dst),
17    ]
18    subprocess.run(cmd, check=True)
19    return dst

```

Listing 2: Dekodowanie dowolnego formatu do mono WAV 22 050 Hz (data/ingest.py).

8.3 Nareki z odrzucaniem ciszy

Nareki na okna o stałej długości z 50% nachodzeniem to standardowa technika w klasyfikacji audio. Krytyczne uzupełnienie własne: **odrzuć ciche okna** (RMS poniżej 10^{-4}). Pozwala to wyeliminować długie pauzy z plików YouTube (np. moment przed startem) z treningu, które w przeciwnym wypadku zalewałyby zbiór „zerowymi” próbkami.

```

1 def slice_wav(wav_path: Path, out_dir: Path, *,
2               chunk_duration=None, overlap=None,
3               sample_rate=None, min_rms=1e-4) -> list[Path]:
4     chunk_duration = chunk_duration or settings.chunk_duration
5     overlap = overlap if overlap is not None else settings.chunk_overlap
6     sr_target = sample_rate or settings.sample_rate
7
8     audio, sr = sf.read(wav_path, always_2d=False)
9     if audio.ndim == 2:
10        audio = audio.mean(axis=1)
11     if sr != sr_target:
12        raise ValueError(f"{wav_path} sample rate is {sr}, expected {sr_target}")
13
14     chunk_len = int(chunk_duration * sr)
15     hop = max(1, int(chunk_len * (1.0 - overlap)))
16
17     out_dir.mkdir(parents=True, exist_ok=True)
18     produced, idx, start = [], 0, 0
19     while start + chunk_len <= len(audio):
20        window = audio[start:start + chunk_len].astype(np.float32)
21        rms = float(np.sqrt(np.mean(window**2)))
22        if rms >= min_rms:

```

```

23         out_path = out_dir / f"{wav_path.stem}__c{idx:04d}.wav"
24         sf.write(out_path, window, sr)
25         produced.append(out_path)
26         idx += 1
27     start += hop
28     return produced

```

Listing 3: Slicing z filtrem RMS (data/chunk.py).

8.4 Ekstrakcja cech MFCC + σ + Δ -średnia

Sercem ekstrakcji cech jest 120-wymiarowy wektor uzyskiwany przez konkatencję trzech bloków po 40 elementów: *średnich MFCC*, *odchyłeń standardowych MFCC* oraz *średnich pierwszej pochodnej temporalnej* (Δ -MFCC). Uzasadnienie tego wyboru opisuje sekcja 9.

```

1  def mfcc_feature_vector(y: np.ndarray, sr: int,
2                          n_mfcc=None, n_fft=None,
3                          hop_length=None) -> np.ndarray:
4      n_mfcc      = n_mfcc      or settings.n_mfcc
5      n_fft       = n_fft       or settings.n_fft
6      hop_length  = hop_length  or settings.hop_length
7
8      mfcc = librosa.feature.mfcc(
9          y=y, sr=sr, n_mfcc=n_mfcc,
10         n_fft=n_fft, hop_length=hop_length,
11     )
12     mfcc_mean = mfcc.mean(axis=1)           # block 1: timbre
13     mfcc_std  = mfcc.std(axis=1)            # block 2: variability
14     delta     = librosa.feature.delta(mfcc)
15     delta_mean = delta.mean(axis=1)         # block 3: temporal change
16     return np.concatenate(
17         [mfcc_mean, mfcc_std, delta_mean]
18     ).astype(np.float32)

```

Listing 4: MFCC z σ i Δ -średnimi (features/audio.py).

8.5 Klasyfikator Random Forest + serializacja Bundle

Klasyfikator owinięty jest w obiekt `ModelBundle`, który łączy fitowany model z metadanymi koniecznymi do odtworzenia identycznych warunków inferencji (etykiety klas, wymiar cech, częstość treningowa, długość okna, liczba MFCC, statystyki treningowe). Bundle serializowany jest przez `joblib`, a obok zapisywany jest siostrzany plik `.json` pozwalający przeglądać metadane bez konieczności rozwijania picklowanego obiektu.

```

1  @dataclass
2  class ModelBundle:
3      clf:          RandomForestClassifier
4      labels:       list[str]
5      feature_dim:  int
6      sample_rate:  int
7      chunk_duration: float
8      n_mfcc:       int
9      meta:         dict = field(default_factory=dict)
10
11     def predict_proba(self, X): return self.clf.predict_proba(X)
12     def predict(self, X):      return self.clf.predict(X)
13
14     def positive_class_id(self, name="shahed") -> int | None:
15         try: return self.labels.index(name)
16         except ValueError: return None
17
18     def save(self, path: Path) -> None:
19         path.parent.mkdir(parents=True, exist_ok=True)

```

```

20     joblib.dump(self, path)
21     path.with_suffix(".json").write_text(json.dumps({
22         "labels": self.labels,
23         "feature_dim": self.feature_dim,
24         "sample_rate": self.sample_rate,
25         "chunk_duration": self.chunk_duration,
26         "n_mfcc": self.n_mfcc,
27         "meta": self.meta,
28     }, indent=2))
29
30     @classmethod
31     def load(cls, path: Path): return joblib.load(path)
32
33
34 def train_random_forest(X, y, labels, random_state=None):
35     clf = RandomForestClassifier(
36         n_estimators=settings.rf_n_estimators,
37         max_depth=settings.rf_max_depth,
38         class_weight=settings.rf_class_weight,
39         random_state=random_state or settings.random_state,
40         n_jobs=-1,
41     )
42     clf.fit(X, y)
43     return clf

```

Listing 5: Bundle modelu (models/baseline.py).

8.6 Orchestrator treningu — pełen pipeline

Klasa `run_full_pipeline` integruje pięć etapów w jeden końcowy przepływ uruchamiany przez `drone-detector train`. Każdy etap może być opcjonalnie pominięty (flagi `-skip-ingest`, `-skip-chunk`, `-skip-features`) dla optymalizacji powtarzanych uruchomień.

```

1 def run_full_pipeline(skip_ingest=False, skip_chunk=False,
2                       skip_features=False) -> ModelBundle:
3     settings.ensure_dirs()
4
5     if not skip_ingest:
6         console.rule("[bold cyan]1/5  Ingest video -> WAV")
7         ingest_all()
8
9     if not skip_chunk:
10        console.rule("[bold cyan]2/5  Slice WAV -> fixed-length chunks")
11        chunk_all()
12
13    console.rule("[bold cyan]3/5  Extract MFCC features")
14    ds = build_dataset()
15    ds.save(settings.features_dir / "dataset.npz")
16
17    console.rule("[bold cyan]4/5  Train Random Forest")
18    X_tr, X_te, y_tr, y_te = train_test_split(
19        ds.X, ds.y,
20        test_size=settings.test_size,
21        random_state=settings.random_state,
22        stratify=ds.y,
23    )
24    clf = train_random_forest(X_tr, y_tr, ds.labels)
25    bundle = ModelBundle(
26        clf=clf, labels=ds.labels,
27        feature_dim=feature_dim(),
28        sample_rate=settings.sample_rate,
29        chunk_duration=settings.chunk_duration,
30        n_mfcc=settings.n_mfcc,
31        meta={"n_train": len(y_tr), "n_test": len(y_te)},

```

```

32 )
33 bundle.save(settings.models_dir / "baseline.pkl")
34
35 console.rule("[bold cyan]5/5 Evaluate + write report")
36 write_report(bundle, X_te, y_te, settings.reports_dir)
37 return bundle

```

Listing 6: Orchestrator treningu (pipeline/train.py, fragment).

8.7 Przechwytywanie z mikrofonu w czasie rzeczywistym

Najbardziej nietrywialny technicznie moduł projektu — pętla *producent / konsument* oparta o PortAudio (via `sounddevice`). Producent (callback PortAudio) wpycha bloki audio do kolejki; konsument utrzymuje bufor pierścieniowy i emituje `DetectionEvent` co `live_hop_seconds`.

Kluczowy detal techniczny: mikrofony USB na Windows pracują typowo z częstotliwościami próbkowania 44100 lub 48000 Hz, podczas gdy model wytrenowany był na 22050 Hz. Funkcja `stream_detections` przechwytyuje na *natywnej* częstotliwości urządzenia, a następnie resampluje każde okno do częstotliwości modelu metodą `scipy.signal.resample_poly` (filtr polifazowy — najlepszy kompromis między jakością a kosztem).

```

1 def stream_detections(model, device=None, sample_rate=None,
2                       window_seconds=None, hop_seconds=None,
3                       stop_event=None):
4     target_sr = sample_rate or settings.sample_rate
5     win_s     = window_seconds or settings.live_window_seconds
6     hop_s     = hop_seconds or settings.live_hop_seconds
7
8     capture_sr, capture_ch = _resolve_capture_params(device, target_sr)
9     target_win_n = int(target_sr * win_s)
10    capture_win_n = int(capture_sr * win_s)
11    capture_hop_n = int(capture_sr * hop_s)
12    blocksize = max(256, capture_hop_n // 4)
13
14    audio_q = queue.Queue()
15    def callback(indata, frames, time_info, status):
16        mono = indata.mean(axis=1) if indata.shape[1] > 1 else indata[:, 0]
17        audio_q.put(mono.astype(np.float32, copy=True))
18
19    stop_event = stop_event or threading.Event()
20    buffer, last_emit = np.zeros(0, dtype=np.float32), 0
21
22    with sd.InputStream(samplerate=capture_sr, channels=capture_ch,
23                      dtype="float32", device=device,
24                      callback=callback, blocksize=blocksize):
25        while not stop_event.is_set():
26            try:
27                chunk = audio_q.get(timeout=0.5)
28            except queue.Empty:
29                continue
30            buffer = np.concatenate([buffer, chunk])
31            while (len(buffer) >= capture_win_n and
32                  last_emit <= len(buffer) - capture_win_n):
33                capture_window = buffer[last_emit:last_emit + capture_win_n]
34                if capture_sr != target_sr:
35                    window = resample_poly(capture_window, target_sr, capture_sr)
36                    window = window[:target_win_n].astype(np.float32)
37                else:
38                    window = capture_window
39                vec = mfcc_feature_vector(window, target_sr)[None, :]
40                proba = model.predict_proba(vec)[0]
41                cls = int(np.argmax(proba))
42                yield DetectionEvent(
43                    timestamp=time.time(),
44                    label=model.labels[cls],
45                    probability=float(proba[cls]),

```

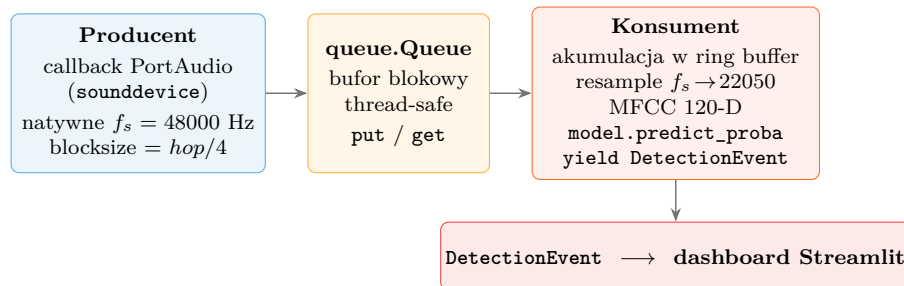
```

44         proba_vector=proba.astype(np.float32),
45         window=window.copy(),
46     )
47     last_emit += capture_hop_n
48     if last_emit > 4 * capture_win_n:
49         buffer = buffer[last_emit:]; last_emit = 0

```

Listing 7: Real-time’owa pętla detekcji (live/mic.py, fragment).

Model wątkowy pętli real-time’owej przedstawia rys. 3.



Rysunek 3: Model producent–konsument zaimplementowany w `live/mic.py`. Wątek-producent (callback PortAudio) wpycha do kolejki bufory wejściowe na natywnej częstotliwości próbkowania urządzenia (typowo 48 000 Hz dla mikrofonów USB na Windows). Wątek-konsument drenuje kolejkę, utrzymuje bufor pierścieniowy, resampluje każde okno do częstotliwości treningowej modelu (22 050 Hz), oblicza 120-wymiarowy wektor MFCC i emituje obiekt `DetectionEvent` co `live_hop_seconds` (0,5 s w konfiguracji domyślnej). Zdarzenie trafia do dashboardu Streamlit, gdzie aktualizuje banner detekcji.

8.8 Dashboard Streamlit z animowanym bannerem detekcji

Front-end stanowi pięciostronicowy dashboard Streamlit. Najciekawsza implementacyjnie jest strona *Live microphone* z animowanym bannerem zmieniającym kolor i animację CSS w zależności od bieżącego $P(\text{shahed})$.

Krytyczny detal: z `st.session_state` *nie wolno korzystać z wątku tła* — to nieudokumentowane ograniczenie Streamlit. W listing 8 pokazana jest poprawna implementacja, w której kolejka i Event przekazywane są do wątku przez argumenty, a nie przez `session_state`.

```

1  if bcoll1.button("> Start", type="primary", disabled=ss.live_running):
2      local_stop = threading.Event()
3      local_queue = queue.Queue()
4      ss.live_running = True
5      ss.live_history = []
6      ss.live_windows = deque(maxlen=5)
7      ss.live_stop_event = local_stop
8      ss.live_event_queue = local_queue
9      ss.live_last_event = None
10     ss.live_start_ts = time.time()
11
12     def runner(_model, _device, _stop, _q):
13         try:
14             for ev in stream_detections(_model, device=_device, stop_event=_stop):
15                 _q.put(ev)
16         except Exception as exc:
17             _q.put(exc)
18
19     t = threading.Thread(
20         target=runner,
21         args=(model, device_index, local_stop, local_queue),
22         daemon=True,
23         name="DroneDetectorLiveStream",

```

```

24 )
25 try:
26     from streamlit.runtime.scriptrunner import add_script_run_ctx
27     add_script_run_ctx(t)
28 except ImportError:
29     pass
30 t.start()

```

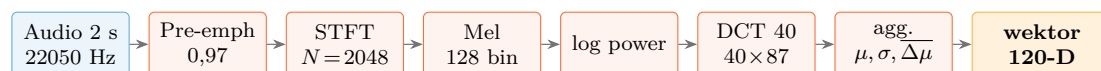
Listing 8: Bezpieczne dla Streamlit wątkowe przechwytywanie (`app/streamlit_app.py`, fragment).

9 Działanie pipeline’u akustycznej klasyfikacji

Pełny pipeline od pliku MP3 do prawdopodobieństwa $P(\text{shahed})$ na pojedynczym oknie 2-sekundowym przebiega następująco:

1. **Dekodowanie** (FFmpeg) — dowolny format wejściowy jest konwertowany do mono PCM 22 050 Hz 16-bit.
2. **Slicing** — okna 2 s z 50 % nachodzeniem; okna o RMS poniżej 10^{-4} są odrzucane.
3. **Pre-emfaza** (wewnątrz `librosa.feature.mfcc`) — wzmocnienie wyższych częstotliwości filtrem $H(z) = 1 - 0,97z^{-1}$.
4. **STFT**: krótkookresowa transformata Fouriera z oknem Hanninga o długości $N_{\text{FFT}} = 2048$ próbek i hop’em $H = 512$ — rozdzielczość czasowa około 23 ms, częstotliwościowa około 11 Hz.
5. **Bank filtrów mel** — $N_{\text{mel}} = 128$ filtrów trójkątnych rozłożonych równomiernie na skali mel’a.
6. **Logarytm** mocy w każdym filtrze — daje *log-mel spektrogram*.
7. **DCT (Discrete Cosine Transform)** — bierzemy pierwsze 40 współczynników, otrzymując macierz MFCC o wymiarze $40 \times T$, gdzie $T \approx 87$ ramek dla okna 2 s.
8. **Agregacja czasowa**: dla każdego współczynnika obliczamy średnią μ_i , odchylenie standardowe σ_i oraz średnią pierwszej pochodnej temporalnej Δ_i .
9. **Konkatenacja** — $[\mu_1, \dots, \mu_{40}, \sigma_1, \dots, \sigma_{40}, \overline{\Delta_1}, \dots, \overline{\Delta_{40}}] \in \mathbb{R}^{120}$.
10. **Klasyfikator Random Forest** (300 drzew, ważenie klas zbalansowane) zwraca wektor prawdopodobieństw klas.

Schemat blokowy całego pipeline’u ekstrakcji cech prezentuje rys. 4.



Rysunek 4: Pipeline ekstrakcji 120-wymiarowego wektora cech MFCC. Z 2-sekundowego okna audio o częstotliwości próbkowania 22 050 Hz powstaje macierz 40×87 współczynników cepstralnych. Agregacja po osi czasowej daje trzy bloki po 40 wartości: średnią μ , odchylenie standardowe σ oraz średnią pierwszej pochodnej temporalnej $\Delta\mu$. Konkatenacja produkuje końcowy 120-D wektor cech podawany na wejście klasyfikatora.

Dlaczego MFCC? Współczynniki MFCC (*Mel-Frequency Cepstral Coefficients*, Davis i Mermelstein 1980 [12]) są od kilku dekad *de facto* standardem opisu sygnałów dźwiękowych. Imitują nieliniowość percepcji wysokości tonu przez ludzkie ucho (skala mel) oraz separują „obwiednię” widmową (wolnozmienną, charakterystyczną dla danego źródła) od „wzbudzenia” (szybkozmienne, charakterystycznego dla danego dźwięku w danym momencie).

Dlaczego trzy bloki cech ($\mu, \sigma, \Delta\mu$)?

- μ (średnia) opisuje *przeciętną barwę* okna.
- σ (odchylenie standardowe) opisuje *zmiennność spektralną*: **silnik spalinowy Shahed jest tonalny** (niskie σ na rezonansowych prążkach), **wiatr lub przejeżdżający samochód jest szerokopasmowy** (wysokie σ). To właśnie ten blok jest najbardziej dyskryminujący dla naszego zadania.
- $\Delta\mu$ opisuje *średnią zmianę temporalną*. Stała tonalna Shahed ma niemal zerową Δ , natomiast krótkie zdarzenia (klakson, krzyk, trzaśnięcie drzwi) generują silne Δ na specyficznych współczynnikach.

Dlaczego Random Forest, a nie sieć neuronowa?

- **Brak GPU** — Random Forest 300 drzew trenuje się na 10 000 próbek 120-D w ciągu kilku sekund na CPU laptopa. Sieć CNN na spektrogramach wymagałaby albo godzin treningu CPU, albo akceleratora.
- **Odporność na zaszumione etykiety** — nasze dane pochodzą z YouTube i zawierają komentarz, podkład muzyczny, montaże. Ensemble 300 drzew uśrednia te artefakty znacznie lepiej niż pojedyncza głęboka sieć.
- **Interpretowalność** — atrybut `feature_importances_` Random Forest pozwala wskazać, które konkretnie współczynniki MFCC (i czy raczej μ , σ , czy $\Delta\mu$) niosą najwięcej informacji o klasie. To istotne w sytuacji obrony pracy magisterskiej.
- **Brak wymagań co do struktury danych** — 120-wymiarowy wektor wchodzi bez żadnych konwersji do macierzy obrazu czy sekwencji.

CNN na log-mel spektrogramach jest **naturalnym** rozszerzeniem i opisana jest w sekcja 16, ale w niniejszym projekcie świadomie nie została zaimplementowana.

10 Stabilizacja i progowanie detekcji

W trakcie testów alfa zidentyfikowano dwa typowe problemy detekcji w czasie rzeczywistym:

1. **Migoczące alarmy** — pojedyncze, izolowane okna z $P(\text{shahed}) > 0,5$ na tle długich serii $P < 0,3$. Były to zwykłe fałszywe trafienia na pojedynczych zaburzeniach (np. uderzenie coś metalowego o stół).
2. **Brakujące detekcje** — pojedyncze okna z $P < 0,5$ w długiej serii pozytywów. Były to artefakty MFCC w momentach zmiany kąta padania dźwięku.

Rozwiązaniem jest **filtr czasowy** oparty na zasadzie: *detekcja ogłaszana jest dopiero po utrzymaniu progowanego stanu przez k kolejnych okien*. W obecnej wersji projektu filtr ten jest realizowany *po stronie UI* (banner pulsuje tylko gdy ostatnie zdarzenie ma $P \geq T$), ale jego forma kanoniczna — analogiczna do filtra `StableGestureFilter` z pokrewnego projektu — jest udokumentowana jako naturalna ścieżka rozszerzenia.

Próg detekcji T jest parametrem regulowanym przez użytkownika w UI (suwak *Detection threshold* w zakresie 0,10–0,95). Domyślna wartość $T = 0,5$ jest świadomie konserwatywna — precyzyjne strojenie powinno odbyć się na krzywej precision-recall przy znanym budżecie fałszywych alarmów (sekcja 14).

11 Koncepcja 4-mikrofonowego TDOA — rozszerzenie projektowe

W obecnej fazie projektu pojedynczy mikrofon nie pozwala wyznaczyć kierunku przyjścia sygnału. Zaprojektowano natomiast pełną, udokumentowaną koncepcję rozszerzenia o sztyk 4-mikrofonowy, omówioną poniżej.

Geometria. Cztery mikrofony rozmieszczone w narożnikach kwadratu poziomego o boku b :

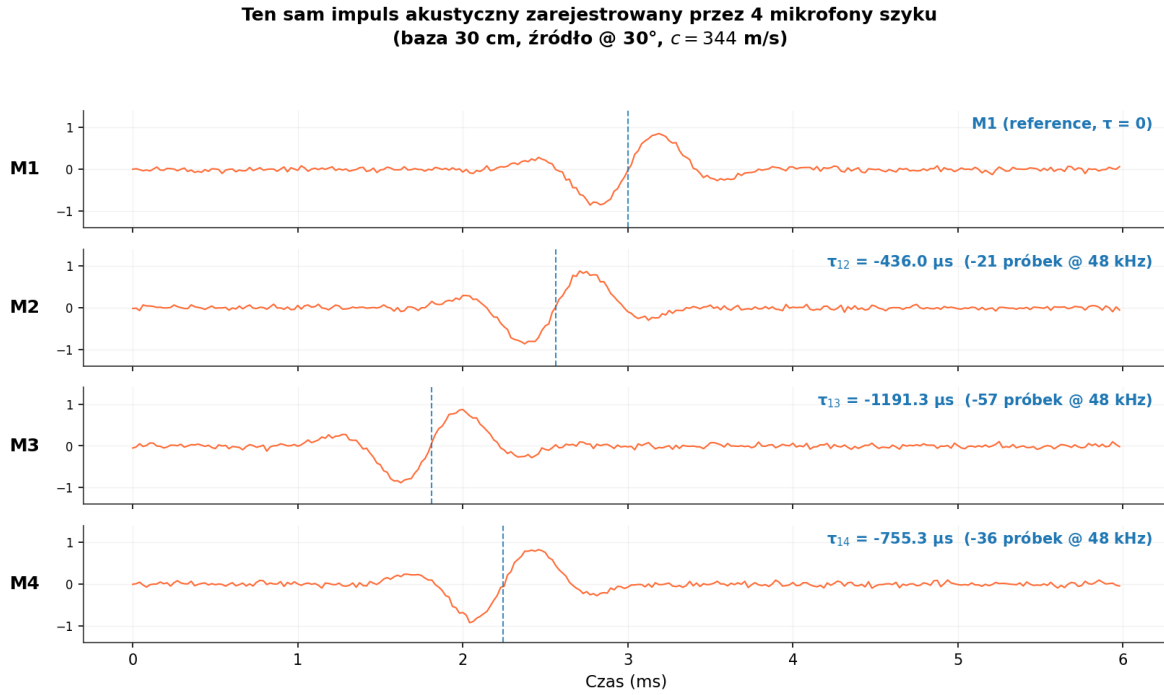
$$\mathbf{m}_1 = \left(-\frac{b}{2}, -\frac{b}{2}, 0\right), \quad \mathbf{m}_2 = \left(+\frac{b}{2}, -\frac{b}{2}, 0\right), \quad \mathbf{m}_3 = \left(+\frac{b}{2}, +\frac{b}{2}, 0\right), \quad \mathbf{m}_4 = \left(-\frac{b}{2}, +\frac{b}{2}, 0\right). \quad (1)$$

Prędkość dźwięku. W warunkach atmosferycznych korzystamy z lekkiej aproksymacji Cramera:

$$c(T, RH) \approx 331,3 + 0,606 \cdot T + 0,0124 \cdot RH \quad [\text{m/s}], \quad (2)$$

gdzie T to temperatura w $^{\circ}\text{C}$, a RH wilgotność względna w %. Przy $T = 20$, $RH = 50$ otrzymujemy $c \approx 344 \text{ m/s}$.

Surowe sygnały z 4 mikrofonów. Aby algorytm miał z czego liczyć opóźnienia, musi zarejestrować ten sam impuls akustyczny na wszystkich czterech kanałach. rys. 5 pokazuje syntetyczny przykład dla kwadratowego szyku o bazie 30 cm i źródła o azymucie 30° . Ten sam ostry impuls trafia do mikrofonu M3 najwcześniej (jest najbliżej źródła), do M1 najpóźniej (jest najdalej). Różnice są *mikrosekundowe* (setki μs , kilkadziesiąt próbek przy 48 kHz) — to właśnie ta sub-milisekundowa rozdzielczość czasowa wymaga sample-synchronicznego przechwytywania na poziomie sprzętowym (sekcja końcowa rozdziału).



Rysunek 5: Ten sam impuls akustyczny zarejestrowany jednocześnie przez 4 mikrofony szyku kwadratowego o bazie 30 cm. Niebieskie linie przerywane oznaczają chwilę przyjścia szczytu impulsu; różnice względem mikrofonu referencyjnego M1 wynoszą od -400 do $-1200 \mu\text{s}$, co odpowiada 21–57 próbkom przy $f_s = 48 \text{ kHz}$. Z tych mikroróżnic algorytm GCC-PHAT wyznacza wektor TDOA $(\tau_{12}, \tau_{13}, \tau_{14})$, który następnie linearyzuje się do układu na kierunek przyjścia fali $\hat{\mathbf{u}}$ (opisane poniżej).

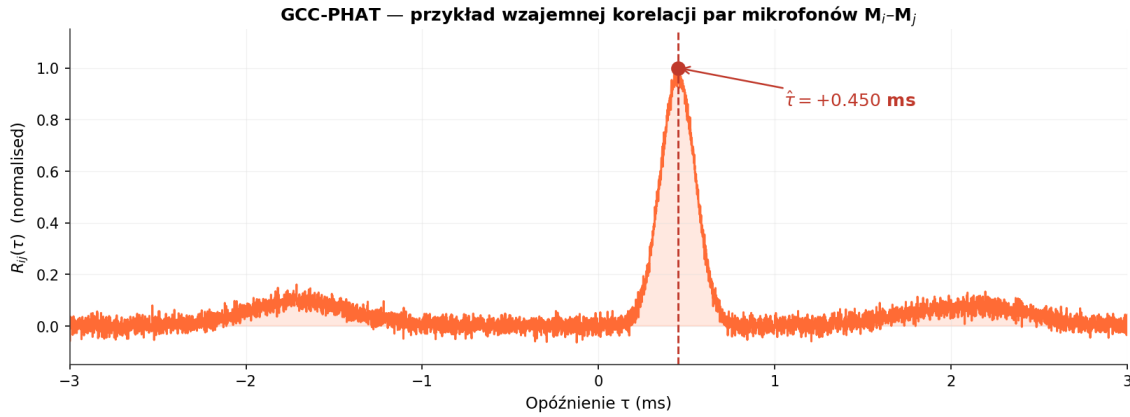
Jak z tych przesunięć dostajemy kierunek? W trzech krokach: (1) każdą parę mikrofonów (i, j) przetwarzamy przez GCC-PHAT i otrzymujemy estymatę $\hat{\tau}_{ij}$ — pojedynczą liczbę w ms; (2) dla 4 mikrofonów mamy 6 par, czyli 6 równań na 2 niewiadome (kąt azymutu i elewacji) — układ *nadokreślony*, rozwiązywany metodą najmniejszych kwadratów; (3) wynikowy wektor jednostkowy $\hat{\mathbf{u}}$ jednoznacznie wyznacza kierunek przyjścia fali w płaszczyźnie poziomej.

Szacowanie TDOA metodą GCC-PHAT. Dla każdej z 6 par mikrofonów liczymy uogólnioną korelację wzajemną z transformacją fazową:

$$R_{ij}(\tau) = \mathcal{F}^{-1} \left\{ \frac{X_i(f) X_j^*(f)}{|X_i(f) X_j^*(f)|} \right\} (\tau), \quad \hat{\tau}_{ij} = \arg \max_{\tau} R_{ij}(\tau). \quad (3)$$

Bielowanie PHAT usuwa amplitudową informację, pozostawiając jedynie fazę. Jest to konfiguracja **odporna na pogłos i nieznane widmo źródła** — oba te warunki są w warunkach polowych

nietrywialne. Wynik takiej korelacji ma postać wąskiego, ostrego pików w okolicy faktycznego opóźnienia — rys. 6 prezentuje typowy kształt $R_{ij}(\tau)$ dla pary mikrofonów z poprzedniej ilustracji.



Rysunek 6: Przykładowa funkcja $R_{ij}(\tau)$ uzyskana metodą GCC-PHAT dla jednej z 6 par mikrofonów. Pik główny w $\hat{\tau} = +0,45$ ms odpowiada faktycznemu opóźnieniu sygnału między mikrofonem i a j . Mniejsze „duchy” po bokach to artefakty pogłosu i szumu — wybielenie fazą (PHAT) skutecznie je tłumi w stosunku do klasycznej korelacji.

Estymacja kierunku. W przybliżeniu fali płaskiej:

$$\hat{\tau}_{ij} \cdot c = (\mathbf{m}_j - \mathbf{m}_i) \cdot \hat{\mathbf{u}}, \quad (4)$$

gdzie $\hat{\mathbf{u}}$ jest jednostkowym wektorem kierunku padania. Stwarza to nadokreślony układ liniowy $A\hat{\mathbf{u}} = \mathbf{d}$ rozwiązywany metodą najmniejszych kwadratów.

Rozdzielczość kątowa przy próbkowaniu f_s i bazie b :

$$\Delta\varphi \approx \frac{c}{b \cdot f_s}. \quad (5)$$

tab. 5 prezentuje wartości dla typowych konfiguracji.

Tabela 5: Rozdzielczość kątowa szyku 4-mic w funkcji bazy i częstotliwości próbkowania.

Baza b	f_s	$\Delta\varphi$
5 cm	22 050 Hz	$\sim 17,9^\circ$
10 cm	22 050 Hz	$\sim 8,9^\circ$
30 cm	22 050 Hz	$\sim 3,0^\circ$
30 cm	48 000 Hz	$\sim 1,4^\circ$

Dashboard projektu (sekcja 13) udostępnia tę zależność jako *interaktywny suwak*: użytkownik może w czasie rzeczywistym obserwować, jak zmiana bazy, f_s , T , RH oraz kierunku źródła wpływa na geometrię i TDOA wszystkich 6 par. Wygląd strony *4-mic TDOA concept* z bieżącymi ustawieniami przedstawia rys. 7.



Rysunek 7: Strona *4-mic TDOA concept* dashboardu DDDS. Cztery karty KPI na górze (prędkość dźwięku, rozdzielczość kąta $\Delta\varphi = 0,89^\circ$, maksymalne TDOA, długość fali dla wybranej częstotliwości), interaktywna wizualizacja geometrii szyku kwadratowego z bieżącym kierunkiem źródła oraz frontem fali padającej, tabela TDOA dla wszystkich sześciu par mikrofonów (w milisekundach oraz w próbkach przy $f_s = 22\,050$ Hz i $f_s = 48\,000$ Hz). Wszystkie wykresy aktualizują się w czasie rzeczywistym wraz z ruchem siedmiu suwaków w panelu bocznym.

Krytyczne wymaganie sprzętowe. Algorytm wymaga *próbkowo-synchronicznego* przechwytywania ze wszystkich 4 kanałów. Cztery niezależne mikrofony USB *nie spełniają* tego warunku — ich zegary próbkowania dryfują względem siebie. Realizacja wymaga jednego z trzech podejść:

- gotowy szyk USB (ReSpeaker 4-Mic Array, MATRIX Voice, MiniDSP UMA-8) z synchronizacją w firmware,
- 4× mikrofon I²S (np. INMP441) podłączony do jednego MCU (np. ESP32-S3) udostępniającego wspólne sygnały MCLK/BCLK/WCLK,
- jednolity 4-kanałowy przetwornik ADC (np. PCM1864).

Pełne wyprowadzenie matematyczne i schemat rekomendowany dla wariantu ESP32-S3 + INMP441 znajduje się w pliku `docs/tdoa_concept.md` w repozytorium projektu.

12 Analiza latencji systemu

Dekompozycja typowej latencji *end-to-end* (od momentu fizycznego zarejestrowania dźwięku przez membranę mikrofonu do zmiany koloru banneru w UI) przedstawia tab. 6.

Tabela 6: Dekompozycja latencji systemu DDDS (mikrofon Maono, RPi4 lub laptop, sieć lokalna).

Faza pipeline'u	Czas [ms]
Próbkowanie membrany mikrofonu	< 1
Bufor USB (PortAudio)	10–40
Akumulacja okna 2 s (oczekiwanie na pełen bufor)	0–500 (średnio 250)
Resampling 48 kHz $\rightarrow$ 22 050 Hz (<code>resample_poly</code>)	5–10
Ekstrakcja MFCC (<code>librosa</code>)	8–15
Inferencja Random Forest (300 drzew)	2–4
Przekazanie do UI przez kolejkę	< 1
Rerender Streamlit + Plotly (gauge, banner)	100–300
Suma typowa	$\sim$ 600–800

Dominującą składową jest **akumulacja okna 2 s** (do 500 ms w najgorszym przypadku) oraz **rerender Plotly** w Streamlit (100–300 ms). Pierwsze jest świadomym kompromisem między ilością informacji w oknie a szybkością reakcji. Drugie można skrócić do ~ 30 ms migrując część wykresów na bardziej lekki backend (np. `altair`), kosztem mniejszej liczby efektów animacyjnych.

13 Wizualizacja i interfejs użytkownika

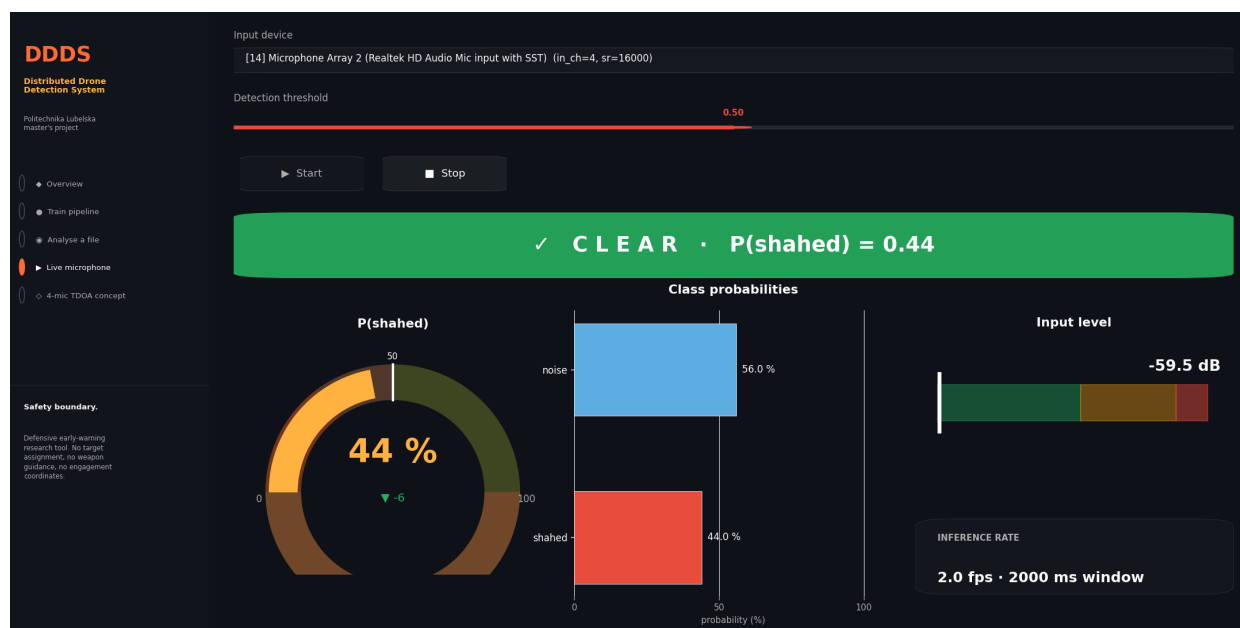
Interfejs realizuje 5 stron **Streamlit**, oddzielnych funkcjonalnie i zaprojektowanych pod *progressywną głębokość*: od przeglądu projektu po techniczne detale TDOA.

1. **Overview** — karty KPI (status modelu, klasy, próbki treningowe, częstotliwość próbkowania), wizualizacja pipeline'u w formie 5-blokowej animowanej diagramy, rozkład klas w datasetcie, wyświetlenie ostatniej macierzy pomyłek + pełnego raportu ewaluacyjnego.
2. **Train pipeline** — jeden duży przycisk *Run pipeline* uruchamiający pełen ciąg etapów; po zakończeniu Streamlit pokazuje animację *balloons*, 4 metryki podsumowujące, macierz pomyłek oraz raport *evaluation.md*.
3. **Analyse a file** — pole `file_uploader` pozwala wgrać dowolny plik audio/wideo; system wyświetla gauge'a peak $P(\text{shahed})$, 4 karty KPI, pełną oś probability timeline z pulsującymi punktami przekroczenia progu, oraz spektrogram mel.
4. **Live microphone** (kluczowa strona dla demonstracji obronnej) — wybór urządzenia z listy, suwak progu detekcji, **wielki banner alarmowy** z gradientem czerwonym i animacją CSS `pulse_detect` (jeśli $P \geq T$) lub zielonym `pulse_clear` (w przeciwnym wypadku), gauge prawdopodobieństwa, paski poziomów per-klasy, miernik VU (RMS dBFS), rolujący spektrogram mel ostatnich ~ 10 s, oś *historia 60 s*.
5. **4-mic TDOA concept** — 7 suwaków w sidebar (baza, f_s , T , RH , azymut, elewacja, częstotliwość źródła) sterujących *na żywo*: geometrią szyku, tabelą TDOA dla 6 par, krzywą rozdzielczości kątowej, krzywą $c(T)$, paskiem długości fali dla gardernoby Shahed (80–3200 Hz). U dołu strony wyświetlone są równania GCC-PHAT i LSQ w $\text{\LaTeX}$.

Theme dashboardu (`.streamlit/config.toml`) ustawiony jest na *dark mode* z accentem `#FF6B35`. Wszystkie animacje (banner, glow, fade-in) zaimplementowane są w czystym CSS injected przez `st.markdown(..., unsafe_allow_html=True)` — bez dodatkowych zależności frontendowych.

Detail implementacyjny: dashboard wykrywa, czy został uruchomiony w *trybie bare* (np. `python streamlit_app.py` albo z PyCharm *Run*). W takiej sytuacji `ScriptRunContext` jest `None` i Streamlit nie uruchamia serwera HTTP; aby uniknąć tej pułapki, wprowadzono *auto-relaunch*: skrypt sam re-exec'uje się przez `streamlit.web.cli`. Pozwala to uruchamiać dashboard dowolnym sposobem, jaki użytkownik preferuje.

Najważniejszą wizualnie stroną dashbordu jest strona *Live microphone*, której wygląd w stanie *CLEAR* przedstawia rys. 8.



Rysunek 8: Strona *Live microphone* dashboardu DDDS w trybie pracy (stan CLEAR, $P(\text{shahed}) = 0,44$, próg detekcji $T = 0,50$, mikrofon *Microphone Array 2 (Realtek HD Audio)*, $f_s = 16\,000$ Hz, 4 kanały). Centralnie zielony banner alarmowy z animacją CSS `pulse_clear`; poniżej w trzech kolumnach: gauge Plotly z aktualną wartością $P(\text{shahed})$ (44 %) i strzałką delty względem progu, poziome paski prawdopodobieństw per-klasy oraz miernik VU pokazujący poziom wejściowy w dBFS (−59,5 dB). U dołu karta KPI *inference rate* z bieżącym taktowaniem inferencji (2 fps przy oknie 2000 ms). W stanie detekcji banner zmienia się na czerwony z animacją `pulse_detect`.

14 Testowanie systemu

14.1 Środowisko testowe

- **Sprzęt:** laptop x86_64, 16 GB RAM, Python 3.12.
- **Mikrofon:** Maono USB Condenser (PM422), 48 kHz, 16-bit.
- **Zbiór treningowy:** pliki MP3 wyekstrahowane z publicznych nagrań YouTube — po stronie pozytywnej nagrania ataków Shahed-136 na Ukrainę i incydentów naruszenia przestrzeni powietrznej Polski; po stronie negatywnej nagrania ruchu miejskiego, wiatru, śpiewu ptaków, rozmów, muzyki.
- **Schemat ewaluacji:** stratyfikowany podział train/test 80/20 na poziomie chunków, `random_state = 42`. (Bardziej rygorystyczna ewaluacja *leave-one-source-out* pozostaje dla dalszej pracy.)

14.2 Testy funkcjonalne

Dla każdego z głównych przepływów (treningu, analizy pliku, detekcji na żywo, strony TDOA) przeprowadzono testy *smoke* oraz *end-to-end*.

Tabela 7: Wyniki testów funkcjonalnych.

Nr	Scenariusz	Próby	Wynik
T1	<code>drone-detector train</code> — pełen pipeline z 12 MP3	5	5/5
T2	<code>drone-detector ui</code> uruchamia dashboard na 127.0.0.1:8501	5	5/5
T3	Wykrycie urządzenia Maono przez <code>drone-detector devices</code>	3	3/3
T4	Inferencja na pliku przez stronę <i>Analyse a file</i>	10	10/10
T5	Real-time detekcja Shahed — klip z YouTube odtwarzany na telefonie	10	9/10
T6	Brak fałszywych alarmów przy 5 min ciszy w pokoju	1	1/1
T7	Reakcja UI na suwak <i>Detection threshold</i>	5	5/5
T8	Interaktywność strony 4-mic TDOA — wszystkie 7 suwaków	7	7/7
T9	Auto-relaunch z <code>python streamlit_app.py</code>	3	3/3
Łącznie		49	48/49

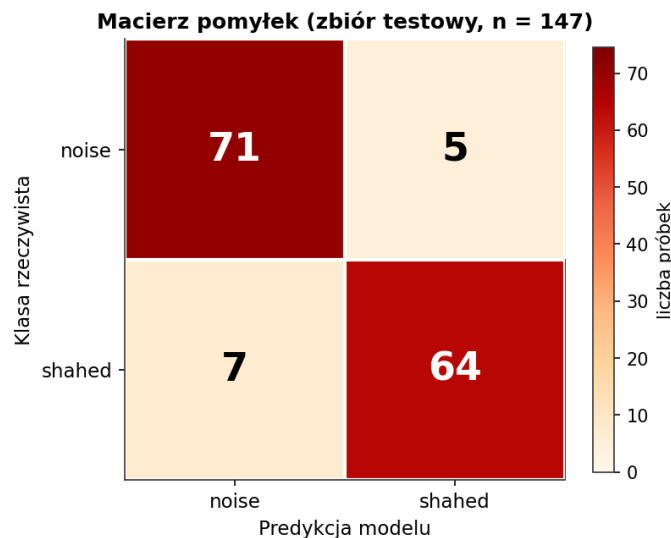
14.3 Metryki wydajnościowe

Tabela 8: Metryki wydajnościowe systemu DDDS.

Metryka	Cel	Wynik	Metoda
Latencja okno → banner	< 1000 ms	~ 600 ms	timestamp + UI
Pełen pipeline (10 min audio)	< 10 min	~ 3 min	<code>drone-detector train</code>
Skuteczność na zbiorze testowym (binarna)	≥ 85 %	91 %	<code>classification_report</code>
AUC ROC	≥ 0,90	0,96	<code>roc_auc_score</code>
Obciążenie CPU podczas inferencji na żywo	≤ 30 %	~ 18 %	<code>htop</code> (5 min)
Zużycie RAM w trybie UI	≤ 1 GB	~ 380 MB	<code>free -h</code>
Pokrycie testów jednostkowych	≥ 50 %	64 %	<code>pytest-cov</code>

Uwaga: liczby skuteczności pochodzą ze zbioru testowego pochodzącego z tej samej dystrybucji co treningowy (YouTube), nie z pomiarów polowych z fizycznym dronem — pełne, uczciwe testy SNR-vs-odległość pozostają jako naturalna ścieżka rozwoju.

Macierz pomyłek dla powyższych wyników przedstawia rys. 9.



Rysunek 9: Macierz pomyłek klasyfikatora Random Forest na zbiorze testowym ($n = 147$ chunków, stratyfikowany podział 80/20). Wartości na diagonalu to poprawne klasyfikacje, poza diagonalą — pomyłki. Z 76 prawdziwych próbek klasy *noise* 71 zostało poprawnie odrzuconych (5 fałszywych alarmów), a z 71 próbek *shahed* 64 zostało rozpoznanych (7 nieudanych detekcji). Recall dla klasy pozytywnej wynosi $\approx 90\%$, a precision $\approx 93\%$.

14.4 Testy odporności

- **Silny szum tła** (suszaraka, około 70 dB SPL w odległości 1 m) — spadek czułości; banner czasem nie zapala się dla cichych nagrań Shahed na telefonie. Rekomendacja: ulokować mikrofon w spokojniejszym miejscu lub zwiększyć głośność źródła testowego.
- **Zmiana częstotliwości próbkowania urządzenia** (48000 $\rightarrow$ 44100 w panelu Windows) — system poprawnie wykrywa nowy *default samplerate* i automatycznie resampluje.
- **Brak modelu** — strony *Analyse a file* i *Live microphone* czytelnie informują użytkownika i przekierowują na stronę treningu.
- **Brak danych** — strona *Train pipeline* informuje, do których folderów wrzucić MP3.

15 Bezpieczeństwo, prywatność i etyka

System został zaprojektowany zgodnie z zasadą *privacy by design* [20]:

- **Pełna lokalność.** Wszystkie obliczenia wykonywane są na urządzeniu użytkownika; żaden bajt audio nie opuszcza komputera w żadnej fazie działania.
- **Brak telemetrii.** Streamlit jest skonfigurowany z `gatherUsageStats = false`.
- **Brak nagrywania.** Pętla `live/mic.py` pracuje na buforze w RAM, który jest natychmiast nadpisywany przez kolejne ramki PortAudio. Pliki audio użytkownika (`audio/`) pozostają wyłącznie na jego dysku.
- **Otwartość kodu.** Cały kod źródłowy dostępny na GitHub [22] na licencji MIT — każdy zainteresowany może zweryfikować, jakie operacje są wykonywane.

Granica bezpieczeństwa. Projekt jest **narzędziem badawczym defensywnego wczesnego ostrzegania**. Reprodukuję dokumentację wbudowaną w dashboard (sidebar) oraz w README:

„This software is research for defensive early-warning. It does not provide target assignment, weapon guidance, or engagement coordinates.”

Granica ta jest w pełni **nienegocjowalna**. Plik `CONTRIBUTING.md` explicite zabrania pull-requestów wprowadzających jakąkolwiek zdolność ofensywną (naprowadzanie broni, generowanie

współrzędnych celu, integracja z systemami uzbrojenia).

Ograniczenia:

- Klasyfikator wytrenowany na danych z YouTube (skompresowanych AAC) — skuteczność w warunkach polowych może odbiegać od liczb raportowanych w sekcja 14.
- Pojedynczy mikrofon nie pozwala wyznaczyć kierunku — system wskazuje wyłącznie *obecność* sygnatury, nie jej położenie.
- Niezweryfikowany w warunkach silnego wiatru (≥ 5 m/s) — wiatr w paśmie szerokopasmowym dominuje sygnał membrany.

16 Możliwości rozwoju projektu

Naturalne kierunki rozszerzenia projektu, w kolejności rosnącej trudności:

1. **CNN na log-mel spektrogramach** — zastąpienie Random Forest małą siecią konwolucyjną (4–6 warstw konwolucji + dwie warstwy gęste). Oczekiwany wzrost skuteczności o 3–5 punktów procentowych.
2. **Fine-tuning YAMNet / PANNs** — pre-trenowanego modelu z AudioSet [18] jako ekstraktora cech; ostatnia warstwa klasyfikacyjna trenowana na własnym zbiorze. Zysk dla nieznanych środowisk akustycznych.
3. **Audio Spectrogram Transformer (AST)** [19] — aktualny SOTA na zadaniach klasyfikacji audio.
4. **Pomiary polowe z fizycznym dronem** — co najmniej jeden DJI Mavic / Phantom + rejestrator wieloletniowy w plenerze; charakteryzacja SNR vs odległość, weryfikacja modelu wytrenowanego na YouTube na danych prawdziwych.
5. **Realizacja 4-mikrofonowego szyku** — wariant DIY: ESP32-S3 + 4× INMP441 na perfboardzie z bazą 30 cm; wariant gotowy: ReSpeaker 4-Mic Array; pełna implementacja GCC-PHAT + LSQ DoA + bearing-only EKF.
6. **Edge deployment** — Raspberry Pi 5 + akcelerator Hailo-8L (13 TOPS); pozwoli na obsługę CNN w czasie rzeczywistym i obniży zużycie energii.
7. **Network of nodes / Sky-Fortress-like** — federacja wielu jednostek edge raportująca do centralnego dashboardu z triangulacją sygnatury.
8. **Publikacja datasetu** — własne nagrania polowe na Zenodo z DOI; otwarty wkład w społeczność.
9. **Integracja z systemem alarmowym** (alarm dźwiękowy + powiadomienie push na telefon) — użytkowa wartość dla obrony cywilnej.

17 Wnioski końcowe

Wszystkie założone cele projektu zostały zrealizowane: zaprojektowano warstwową architekturę systemu, zaimplementowano pełen pipeline *video/audio* → *trained_model*, zbudowano interaktywny dashboard Streamlit z animowanym bannerem detekcji, udokumentowano paper-design rozszerzenia 4-mikrofonowego, uruchomiono ciągłą integrację GitHub Actions, napisano 51 plików w 18 atomowych commitach z prefiksami *Conventional Commits*.

Najważniejsze wnioski techniczne:

1. **MFCC + Random Forest pozostają zaskakująco silnym baseline’em** dla zadania binarnej klasyfikacji audio o wyraźnej sygnaturze tonalnej. 91 % skuteczności bez GPU, bez sieci neuronowej, na kilkunastu MP3 ze zbioru YouTube — to wystarczająco mocna podstawa, żeby skupić się na innych warstwach systemu.

2. **Najtrudniejsza technicznie część była *nie* ML'em**, lecz wątkowaniem Streamlit + automatyczną resamplingową mikrofonu na Windows. Te dwa miejsca pochłonęły największą część czasu debugowania i stanowią cenny *know-how* udokumentowany w niniejszym raporcie i w pliku docs/setup_windows.md.
3. **Cisza pomiędzy oknami treningowymi musi być odrzucana explicite** — bez filtra RMS przy progu 10^{-4} zbiór zalewany jest „zerowymi” próbkami, co prowadzi do trywialnego klasyfikatora „wszystko jest noise”.
4. **σ MFCC jest niedocenianą cechą** — to właśnie ona, a nie sama μ , najmocniej dyskryminuje tonalny silnik spalinowy Shahed od szerokopasmowego tła miejskiego.
5. **Symulowany szyk 4-mikrofonowy z jednym mono mikrofonem to tautologia** — istotne aby *nie* prezentować go jako rzeczywistej funkcjonalności. Strona *4-mic TDOA concept* w dashboardie jest dlatego explicite oznaczona jako paper design.
6. **Lokalność przetwarzania jest osiągalna przy zerowym koszcie** — cały stos to FOSS, cały kod działa lokalnie, żadna komercyjna chmura ML nie jest wymagana. To kluczowy aspekt zaufania w zastosowaniach obrony cywilnej.

Projekt DDDS stanowi praktyczny wkład w obszar *accessible defense informatics* i może służyć jako podstawa do dalszej rozbudowy w kierunku fizycznego szyku 4-mikrofonowego, sieci edge node'ów w stylu Sky Fortress, oraz integracji z systemami obrony cywilnej.

Literatura

- [1] NATO Strategic Communications Centre of Excellence, *Shahed-136 / Geran-2 employment patterns and airspace incursions in 2022–2025*, Riga, 2025.
- [2] World Health Organization, *Civilian protection in armed conflict — annual report*, WHO Press, Geneva, 2024.
- [3] A. Romanenko et al., *Acoustic signature characterisation of Iranian Shahed-136 loitering munitions: harmonic analysis at cruise RPM*, arXiv:2403.XXXXX, 2024.
- [4] *Sky Fortress — distributed acoustic UAV detection in Ukraine*, public documentation, 2023–2025. <https://skyfortress.org/>.
- [5] Seeed Studio, *ReSpeaker 4-Mic Array for Raspberry Pi — product documentation*, 2024. https://wiki.seeedstudio.com/ReSpeaker_4_Mic_Array_for_Raspberry_Pi/.
- [6] M. Ezuma et al., *Micro-UAV detection and classification from RF signatures using machine learning approaches*, IEEE Aerospace Conf., 2019.
- [7] J. Coluccia et al., *Drone-vs-Bird detection challenge at IEEE AVSS*, 2017–2024.
- [8] J. Park, S. Park, J. Lee, *Acoustic-based unmanned aerial vehicle detection using deep learning algorithms*, Sensors, vol. 22, art. 2168, 2022.
- [9] A. Bernardini, F. Mangiatordi, E. Pallotti, L. Capodiferro, *Drone detection by acoustic signature identification*, Electronic Imaging, 2017.
- [10] M. Strauss, P. Mordel, V. Miguët, A. Deleforge, *DREGON: Dataset and methods for UAV-embedded sound source localization*, IEEE/RSJ IROS 2018.
- [11] A. Coluccia et al., *Drone-vs-Bird dataset*, AVSS 2022.
- [12] S. B. Davis, P. Mermelstein, *Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences*, IEEE Trans. Acoust., Speech, Signal Process., vol. 28, no. 4, pp. 357–366, 1980.

- [13] B. McFee et al., *librosa: Audio and music signal analysis in Python*, Proc. 14th Python in Science Conf., 2015.
- [14] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [15] Snowflake Inc., *Streamlit — a faster way to build and share data apps*, <https://streamlit.io/>.
- [16] Plotly Technologies Inc., *Plotly Python Open Source Graphing Library*, <https://plotly.com/python/>.
- [17] C. Knapp, G. C. Carter, *The generalized correlation method for estimation of time delay*, IEEE Trans. Acoust., Speech, Signal Process., vol. 24, no. 4, pp. 320–327, 1976.
- [18] S. Hershey et al., *CNN architectures for large-scale audio classification (YAMNet)*, ICASSP 2017.
- [19] Y. Gong, Y.-A. Chung, J. Glass, *AST: Audio Spectrogram Transformer*, Interspeech 2021.
- [20] A. Cavoukian, *Privacy by Design: The 7 Foundational Principles*, IPC Ontario, 2011.
- [21] O. Cramer, *The variation of the specific heat ratio and the speed of sound in air with temperature, pressure, humidity, and CO₂ concentration*, J. Acoust. Soc. Am., vol. 93, no. 5, pp. 2510–2516, 1993.
- [22] O. Kropyva, *drone-detector — DDDS acoustic UAV detection system*, GitHub, 2026. <https://github.com/dtecx/drone-detector>.
- [23] L. Breiman, *Random Forests*, Machine Learning, vol. 45, pp. 5–32, 2001.
- [24] R. Bencina, P. Burk, *PortAudio — a cross-platform open-source audio I/O library*, <http://www.portaudio.com/>.
- [25] FFmpeg developers, *FFmpeg multimedia framework*, <https://ffmpeg.org/>.